



南开大学
Nankai University

《软件测试与维护》大作业报告

Online-Boutique 运维实验与 ISSRE24-KPIRoot 论文复现

实验标题： Online-Boutique 运维实验与 ISSRE24-KPIRoot 论文复现

课程名称： 软件测试与维护

报告范围： 组内两篇论文复现任务中的一篇：ISSRE24-KPIRoot

微服务系统： JoinFyc/Online-Boutique

参与者： 王新杰（学号：2311901）；王子祺（学号：2312385）

GitHub： [https://github.com/Sinclair987/
software-testing-final-project](https://github.com/Sinclair987/software-testing-final-project)

指导教师： 张圣林

完成日期： 2026-06-06

说明：课程大作业以小组形式完成，基本要求包含两篇论文复现。本报告仅记录组内其中一篇论文 ISSRE24-KPIRoot 的复现工作；另一篇论文复现与智能体封装由组内其他成员另行整合。

目录

摘要	3
1 作业范围与系统概况	3
2 阶段一：微服务系统部署	4
3 阶段二：监控、故障注入与数据采集	4
3.1 监控指标与告警 KPI 设计	4
3.2 故障场景与数据格式	5
4 阶段三：Selenium 与 JMeter 测试	6
5 ISSRE24-KPIRoot 论文方法	7
5.1 原论文问题定义	7
5.2 核心算法流程	8
5.3 本项目适配策略	8
6 阶段四：KPIRoot 复现实验结果	9
6.1 实验设置	9
6.2 主实验结果	9
6.3 消融实验与解释	11
7 差异、局限与优化方向	12
8 参与者与贡献	12
9 总结	13
参考资料	13

摘要

本报告围绕 JoinFyc/Online-Boutique 开源微服务系统，记录本地部署、监控维护、故障注入、黑盒测试以及 ISSRE 2024 论文 KPIRoot 的复现工作。实验使用 Minikube 部署 Online-Boutique，使用 Prometheus、Grafana、Blackbox Exporter 与 ChaosMesh 采集正常与故障状态下的时序 KPI 数据，使用 Selenium 和 JMeter 验证系统功能与性能，最后基于采集数据实现 KPIRoot 中的 PAA、SAX、相似度分析、Granger 风格因果得分和综合排序。实验结果显示，在 paymentservice CPU Stress、frontend CPU Stress 和 paymentservice Pod Kill 三组故障数据中，复现算法均能将真实根因服务排在第一位。

关键词：Online-Boutique；Prometheus；Grafana；ChaosMesh；Selenium；JMeter；KPIRoot；根因定位

1 作业范围与系统概况

课程要求围绕微服务系统完成部署、测试、维护和论文复现，并在报告中展示微服务系统、Selenium/JMeter 测试、ChaosMesh 故障注入、Prometheus/Grafana 监控、异常数据采集与论文算法效果。本项目选择第二档要求中的 Online-Boutique，而不是实验指南中的 SockShop，因此系统规模和服务组成更接近真实云原生应用。

需要特别说明的是，课程要求小组整体复现两篇异常检测或故障诊断相关论文。本报告对应组内其中一篇，即 ISSRE24-KPIRoot。报告中的代码、数据和结果均来自公开仓库：

<https://github.com/Sinclair987/software-testing-final-project>

项目	内容
微服务系统	JoinFyc/Online-Boutique
本地集群	Minikube
命名空间	online-boutique
监控组件	Prometheus、Grafana、Blackbox Exporter
故障注入	ChaosMesh
测试工具	Selenium、JMeter
复现论文	ISSRE 2024 - KPIRoot: Efficient Monitoring Metric-based Root Cause Localization in Large-scale Cloud Systems

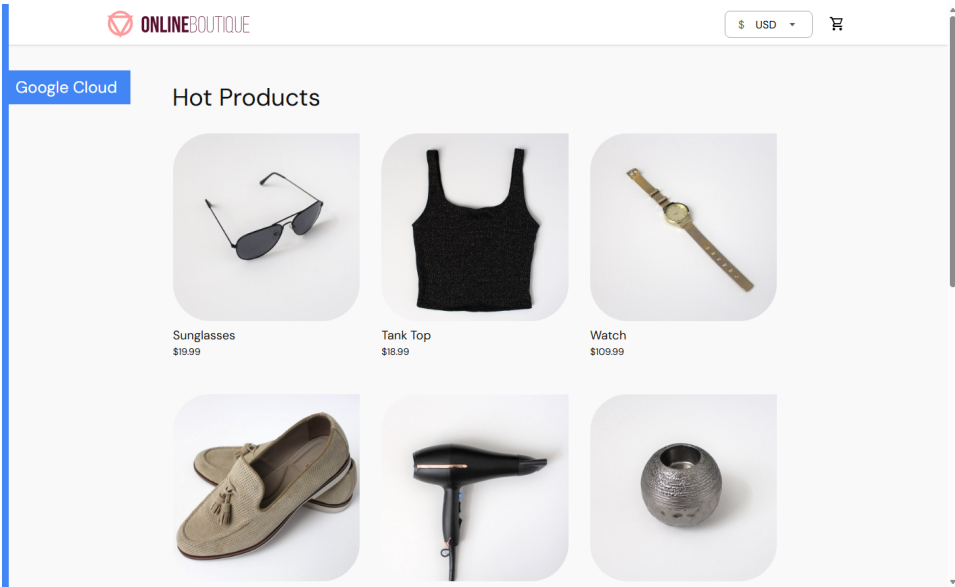


图1 Online-Boutique 前端首页

2 阶段一：微服务系统部署

阶段一完成 Online-Boutique 在 Minikube 中的部署。部署时使用 release/kubernetes-manifests.yaml 中的预构建镜像，并创建 online-boutique 命名空间。部署完成后，12 个业务 Pod 均进入 1/1 Running 状态，frontend 服务通过本地端口转发暴露到 127.0.0.1:8088。

```
kubectl apply -n online-boutique -f .\FinalProject\Online-Boutique\release\kubernetes-manifests.yaml
kubectl wait --for=condition=available deployment --all -n online-boutique --timeout=300s
kubectl port-forward -n online-boutique service/frontend 8088:80
```

直接访问 Minikube NodePort 在 Windows 环境下不够稳定，因此实验中采用 kubectl port-forward 作为稳定前端入口。项目保留 resume-online-boutique.ps1 和 port-forward-frontend.ps1 作为重启后恢复脚本。

3 阶段二：监控、故障注入与数据采集

阶段二复用 monitoring 命名空间中的 Prometheus 与 Grafana，并部署 Blackbox Exporter 对 Online-Boutique 前端进行探测。Prometheus 能采集容器 CPU、内存、文件系统读写、Pod 状态、容器重启次数以及前端 probe_success/probe_duration_seconds 等指标。Grafana Dashboard 用于观察故障前、故障中和恢复后的指标变化。

3.1 监控指标与告警 KPI 设计

组件	作用
Prometheus	采集 Kubernetes、cadvisor、kube-state-metrics、node-exporter 与 Blackbox 指标
Grafana	展示 Online-Boutique Dashboard，包括 CPU、内存、运行状态和探针指标
Blackbox Exporter	提供前端可用性和响应耗时指标，作为服务质量告警 KPI
ChaosMesh	注入 CPU Stress 和 Pod Kill 故障

为了适配 KPIRoot 的输入形式，阶段二将监控指标分为两类：一类是系统级或聚合型告警 KPI，例如前端探针耗时、前端探针成功率、合成总 CPU 和合成总内存；另一类是候选根因 KPI，例如各微服务的 CPU、内存、文件系统读写、运行状态和重启次数。这样设计的原因是，KPIRoot 的定位目标不是判断系统是否异常，而是在一个系统级异常出现后，从底层 KPI 中找出最可能导致异常的对象。

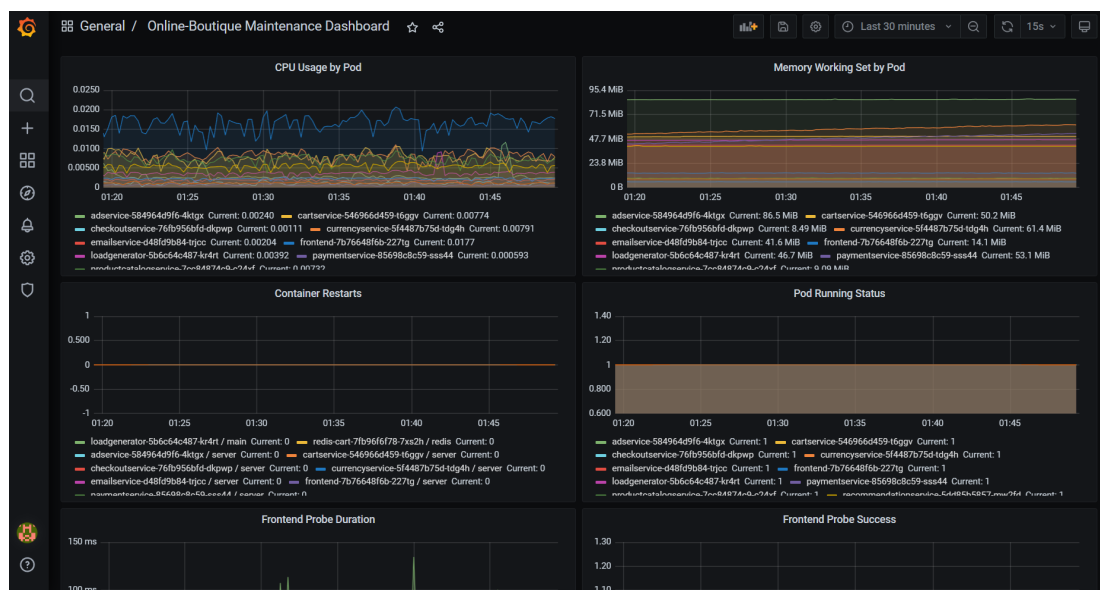


图2 Grafana 正常基线监控面板

3.2 故障场景与数据格式

阶段二最终保留三组故障数据。每组数据均包含 metadata.yaml、Prometheus 原始 query_range 导出、处理后的 kpi_matrix.csv、series_labels.json 和截图证据。

数据集	故障类型	目标服务	矩阵规模	阶段四真实根因
stress-paymentservice-cpu-001	CPU Stress	paymentservice	115 行 × 62 列	cpu__paymentservice
pod-kill-paymentservice-001	Pod Kill	paymentservice	67 行 × 62 列	paymentservice 服务
stress-frontend-cpu-001	CPU Stress	frontend	68 行 × 60 列	cpu__frontend

数据导出采用 15 秒步长，覆盖 baseline、fault 和 recovery 三段窗口。原始 Prometheus 结果先按查询分别保存到 prometheus_raw 目录，再转换为宽表矩阵。转换时将 Pod 名称归并到稳定的服务名，例如 paymentservice-85698c8c59-sss44 和替换后的 paymentservice-85698c8c59-5sx8h 在 processed 矩阵中均映射为 paymentservice 服务相关 KPI。这一处理有助于算法在服务层面进行根因定位，但也会弱化 Pod Kill 中单个 Pod 身份变化，因此报告中将 Pod Kill 作为补充案例。

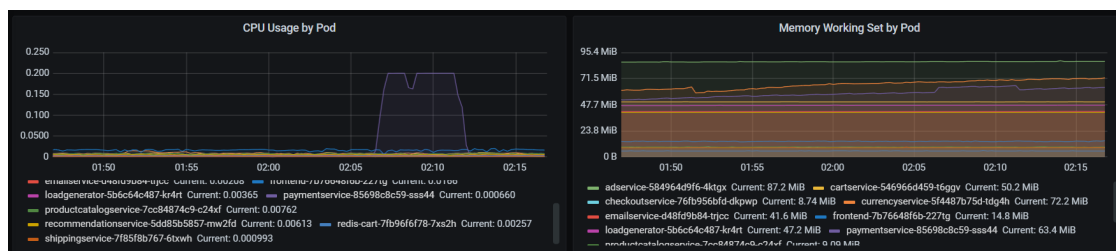


图3 paymentservice CPU Stress 故障期间 CPU 指标升高

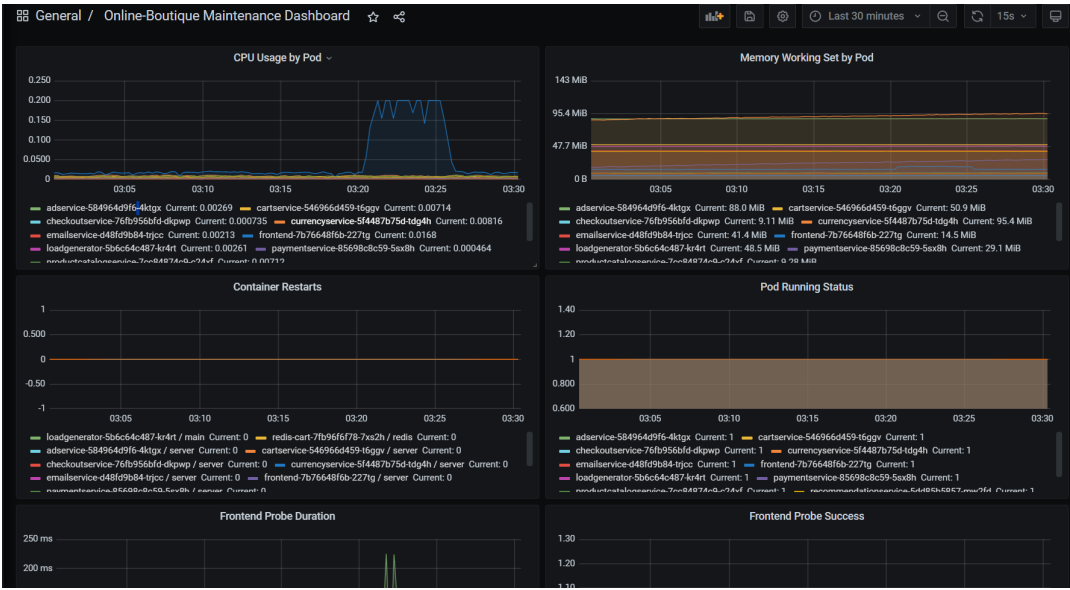


图4 frontend CPU Stress 故障期间 CPU 指标升高

paymentservice CPU Stress 场景中，cpu__paymentservice 从约 0.00064 升至故障窗口平均约 0.16999；frontend CPU Stress 场景中，cpu__frontend 从约 0.016 升至故障窗口平均约 0.170，且前端探针耗时出现升高。Pod Kill 场景中 Kubernetes 快速创建替换 Pod，整体指标变化较弱，但原始 Prometheus 数据保留了旧 Pod 与新 Pod 的身份变化。

4 阶段三：Selenium 与 JMeter 测试

阶段三使用 Selenium 对 Online-Boutique 前端核心购物流程进行功能测试，使用 JMeter 对首页、商品详情、加入购物车、购物车和结算流程进行性能测试。Selenium 运行环境为 Microsoft Edge，JMeter 版本为 5.1.1。

测试用例	指标	类型	时间 ms
test_home_page_lists_products	home_page_load_ms	page_load	1803.1
test_product_detail_page	product_detail_page_load_ms	page_load	1750.6
test_add_to_cart	add_to_cart_interaction_ms	interaction	165.09
test_checkout_flow	checkout_submit_interaction_ms	interaction	228.88

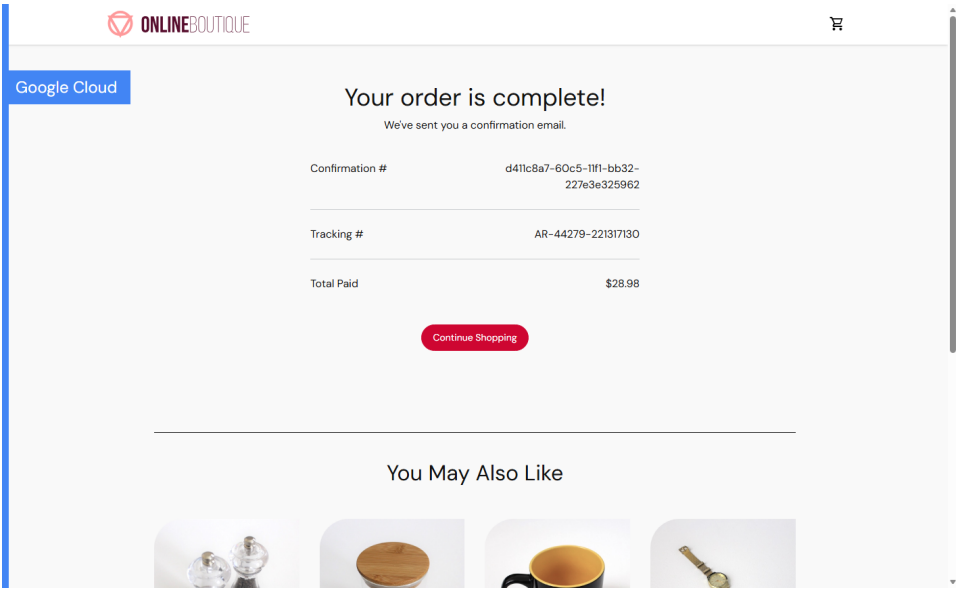


图5 Selenium 结算流程完成页面

Run	Main Samples	Raw Samples	Errors	Error %	Avg ms	P90 ms	P95 ms
smoke-001	5	7	0	0.0	31.8	49.8	51.4
normal-001	250	350	0	0.0	35.83	77.0	83.0
higher-001	500	700	0	0.0	33.53	73.0	78.0

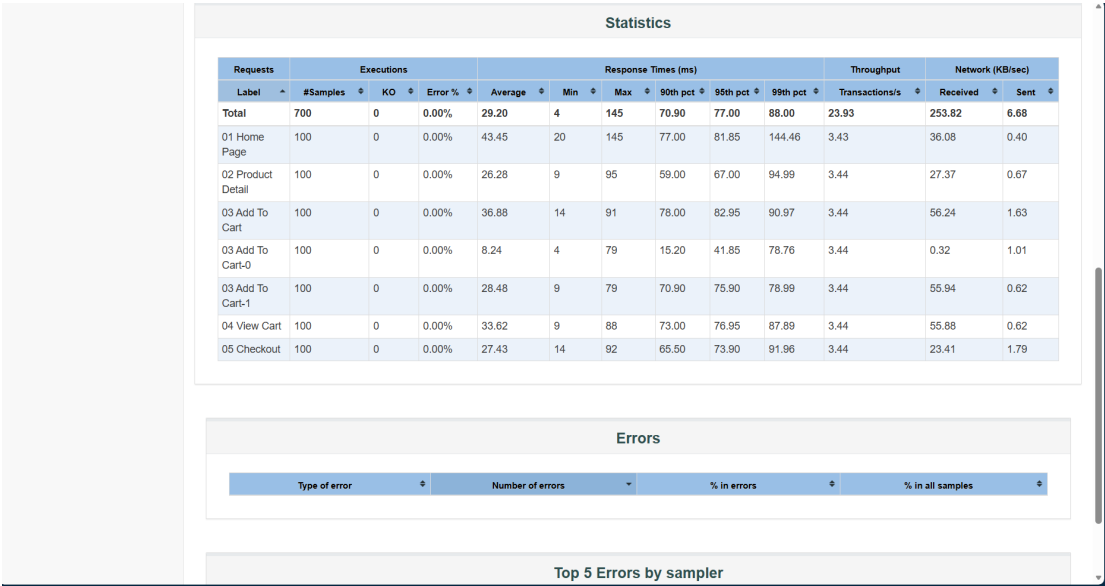


图6 JMeter higher-001 统计结果与错误率

Selenium 共 4 个用例全部通过，页面加载时间约 1.75 到 1.80 秒，关键交互响应时间低于 0.25 秒。JMeter 三组负载均未出现错误，higher-001 中 500 个主采样器的平均响应时间约 33.53 ms，P95 响应时间约 78 ms，说明在本地实验负载下系统保持稳定。

5 ISSRE24-KPIRoot 论文方法

5.1 原论文问题定义

KPIRoot 的目标是在系统级 alarm KPI 出现异常后，从大量底层 KPI 中定位最可能导致异常的根因 KPI。原论文面向 Cloud H 大规模云系统，输入包括主机集群层面的 alarm KPI 和 VM 级别的 CPU、内存、I/O、带宽等 KPI。其核心思想是：真正的根因 KPI 在异常窗口内应当与 alarm KPI 有相似的异常形状，并且其变化应当在时序上能够解释 alarm KPI 的变化。

设系统级告警 KPI 为 X_{alarm} ，候选底层 KPI 为 X_i 。算法需要为每条 X_i 计算一个相关性得分 c_i ，并按得分从高到低输出候选根因。论文强调两个工业可用性要求：一是效率，云系统中 KPI 数量很大，算法必须能在告警后快速完成定位；二是可解释性，运维工程师需要看到为什么某条 KPI 被认为是根因，而不是只得到一个黑盒分类结果。

5.2 核心算法流程

KPIRoot 首先对时间序列进行标准化和 Piecewise Aggregate Approximation（PAA）降维，再使用 Symbolic Aggregate Approximation（SAX）将连续数值序列离散化为符号序列。PAA 降低序列长度，SAX 保留趋势形状并减少噪声影响。在异常窗口内，算法计算 alarm KPI 与每条候选 KPI 的 SAX-Jaccard 相似度，用于衡量两条序列是否具有相似异常形状。

除了形状相似，KPIRoot 还引入 Granger causality 思想。若候选 KPI 的历史值能够显著改善对 alarm KPI 后续值的预测，则说明该候选 KPI 在时序上更可能先于系统级异常发生变化。最后，论文将相似度与因果得分线性组合，得到综合得分并排序。原论文中相似度权重 λ 取 0.9，因果项权重为 0.1。

步骤	含义	本项目实现
标准化	消除不同量纲影响	对每条 KPI 执行 z-score，常数序列置零
PAA	降低时间序列长度	默认保留 32 个 PAA bin，避免课程短序列过度压缩
SAX	将 PAA 数值转为符号	alphabet_size = 9，使用正态分位点作为断点
异常窗口	确定比较区间	优先使用 ChaosMesh 元数据时间窗口
相似度	比较异常形状	SAX multiset-Jaccard
因果得分	衡量时序先后关系	Granger 风格 F 统计量，lag = 2
综合得分	输出排序	$0.9 \times \text{similarity} + 0.1 \times \text{normalized_causality}$

5.3 本项目适配策略

原论文概念	本项目映射
Host/cluster alarm KPI	synthetic_total_cpu、synthetic_total_memory、前端探针耗时与成功率
VM KPI	Online-Boutique 各服务的 CPU、内存、文件系统、运行状态等 KPI
Root-cause VM	被 ChaosMesh 注入故障的目标服务

原论文使用 Cloud H 的主机集群与 VM KPI，本项目使用 Online-Boutique 的服务级 KPI，因此需要进行场景映射。CPU Stress 场景中，将所有服务 CPU 求和得到 synthetic_total_cpu 作为系统级告警 KPI；Pod Kill 场景中，由于服务快速重建且运行状态变化较短，复现中选择 synthetic_total_memory 作为更稳定的告警 KPI。候选 KPI 则排除 alarm_ 和 synthetic_ 前缀，保留服务级 CPU、内存、文件系统、运行状态等指标。

复现代码分为数据读取、算法计算和批量评估三部分。data.py 负责读取阶段二数据、解析 metadata.yaml、构造合成告警 KPI；algorithm.py 实现 PAA、SAX、异常窗口选择、相似度、因果得分和综合排序；cli.py 负责批量运行三组故障、输出 ranking.csv、score_breakdown.csv、summary.csv、消融结果和可视化图。

```
src/kpiroot/algorithm.py
src/kpiroot/data.py
src/kpiroot/cli.py
tests/kpiroot/test_algorithm.py
```

6 阶段四：KPIRoot 复现实验结果

6.1 实验设置

阶段四直接使用阶段二导出的三组故障 KPI 矩阵。每组矩阵的时间戳以 15 秒为间隔，包含故障前基线、故障注入过程和恢复观察窗口。算法运行时读取 metadata.yaml 中的故障时间，映射到 PAA 后的异常窗口；若元数据缺失，则使用趋势比值进行自动异常段检测。评价指标使用 Hit@1、Hit@3、Hit@5 和真实根因服务排名。

参数	取值	说明
paa_size	32	课程数据较短，保留更多 PAA bin 以避免异常窗口过短
alphabet_size	9	与论文示例一致，使用 9 个 SAX 符号表示趋势
granger_lag	2	使用两个历史滞后项估计候选 KPI 对 alarm KPI 的影响
min_segment_bins	8	保证异常窗口内有足够样本用于相似度和因果计算
lambda_weight	0.9	综合得分中相似度权重为 0.9，因果得分权重为 0.1

6.2 主实验结果

故障场景	告警 KPI	真实根因服务	Top-1 KPI	根因服务排名	Hit@1	Hit@3	Hit@5
pod-kill-paymentservice-001	synthetic_total_memory	paymentservice	memory__paymentservice	1	True	True	True
stress-frontend-cpu-001	synthetic_total_cpu	frontend	cpu__frontend	1	True	True	True
stress-paymentservice-cpu-001	synthetic_total_cpu	paymentservice	cpu__paymentservice	1	True	True	True

表中可以看到，三组场景均取得 Hit@1=True。对 CPU Stress 场景而言，系统级 synthetic_total_cpu 与目标服务 CPU 的异常窗口趋势高度一致，因此 Top-1 KPI 分别为 cpu__paymentservice 和 cpu__frontend。Pod Kill 场景中，由于服务级矩阵将替换前后的 Pod 归并为 paymentservice，单个 Pod 消失事件被弱化，算法最终以 memory__paymentservice 作为 Top-1，仍然定位到正确服务。

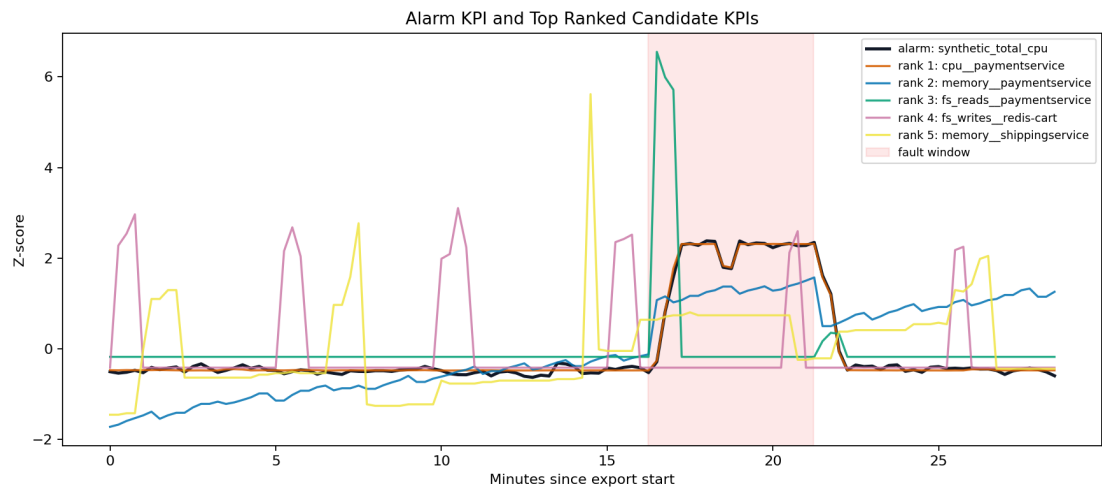


图7 paymentservice CPU Stress：告警 KPI 与 Top 候选 KPI 对比

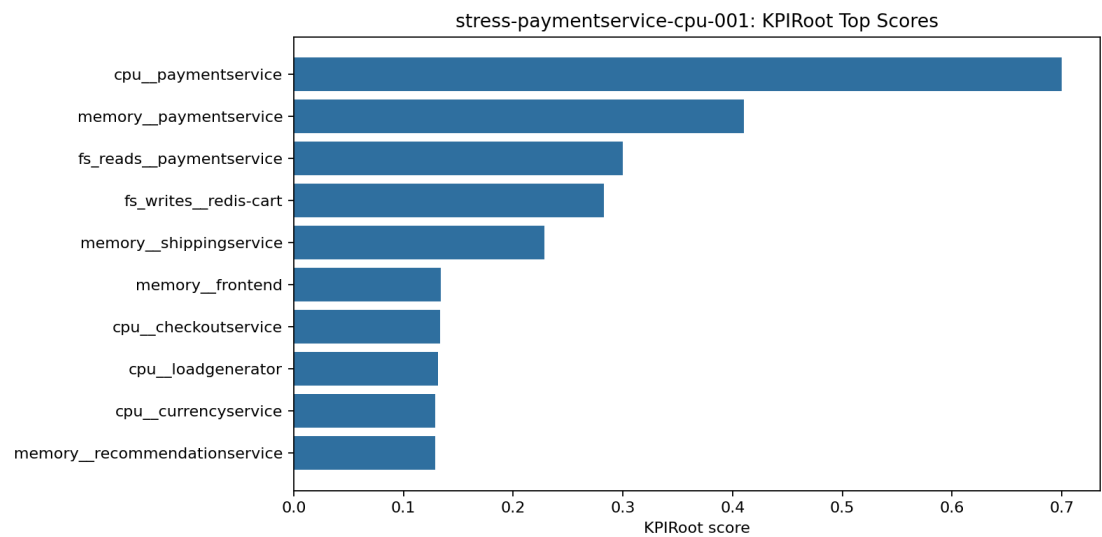


图8 paymentservice CPU Stress：Top-K 根因得分

Rank	KPI	Similarity	Causality	Score
1	cpu__paymentservice	0.778	0.001	0.700
2	memory__paymentservice	0.455	0.014	0.410
3	fs_reads__paymentservice	0.333	0.000	0.300
4	fs_writes__redis-cart	0.231	0.748	0.282
5	memory__shippingservice	0.143	1.000	0.229

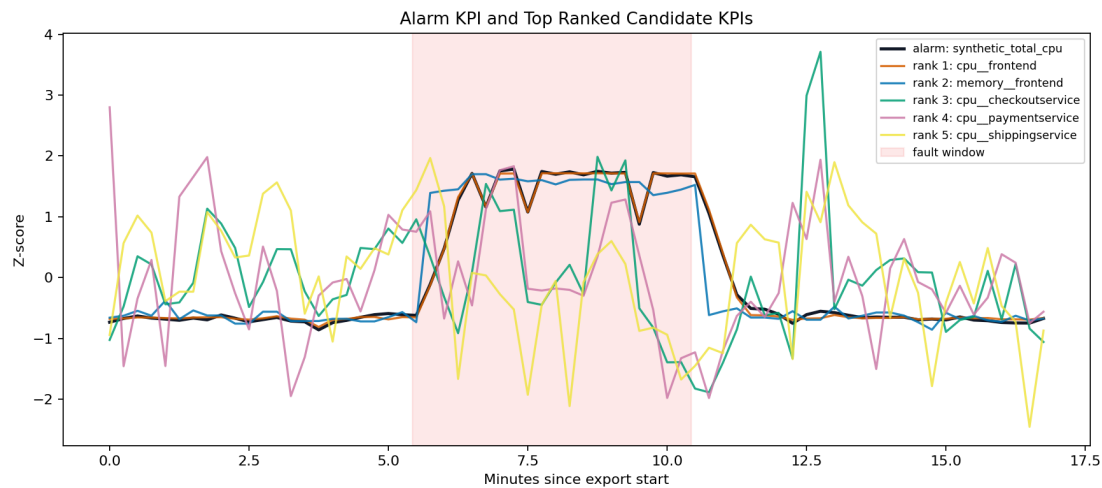


图9 frontend CPU Stress：告警 KPI 与 Top 候选 KPI 对比

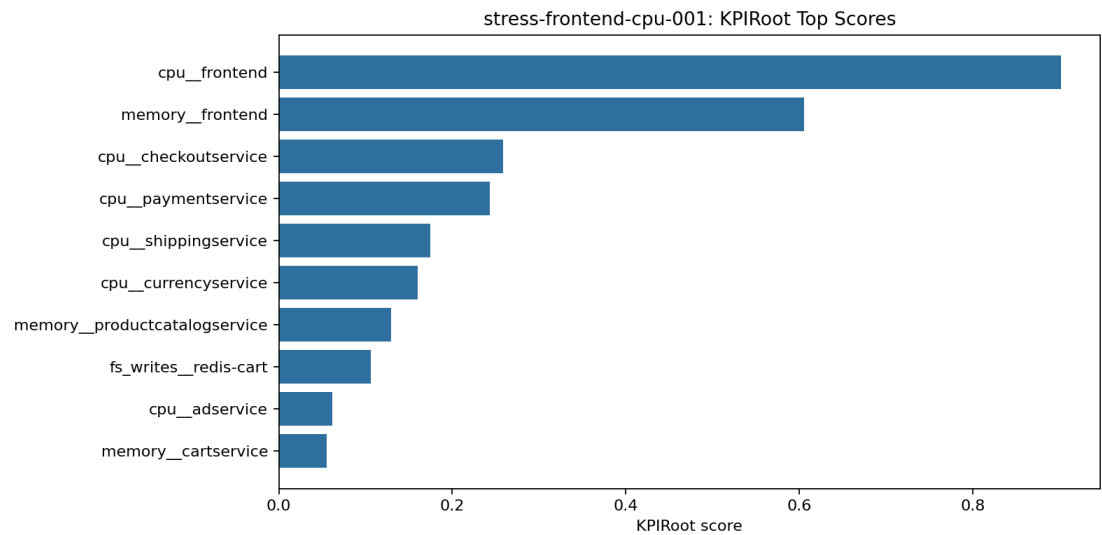


图10 frontend CPU Stress：Top-K 根因得分

Rank	KPI	Similarity	Causality	Score
1	cpu_frontend	1.000	0.020	0.902
2	memory_frontend	0.667	0.060	0.606
3	cpu_checkoutservice	0.176	1.000	0.259
4	cpu_paymentservice	0.250	0.184	0.243
5	cpu_shippingservice	0.111	0.749	0.175

paymentservice CPU Stress 中，cpu__paymentservice 的相似度为 0.778，综合得分约 0.700，明显高于第二名 memory__paymentservice；frontend CPU Stress 中，cpu_frontend 的相似度达到 1.000，综合得分约 0.902。两组结果说明，在故障类型和告警 KPI 对应关系明确的情况下，SAX 相似度能有效捕获根因服务的异常趋势。

6.3 消融实验与解释

故障场景	方法	Top-1 KPI	根因服务排名	Hit@1	Hit@3	Hit@5
pod-kill-paymentservice-001	similarity_only	memory__paymentservice	1	True	True	True
pod-kill-paymentservice-001	causality_only	fs_writes__redis-cart	11	False	False	False
pod-kill-paymentservice-001	kpiroot_combined	memory__paymentservice	1	True	True	True
stress-frontend-cpu-001	similarity_only	cpu__frontend	1	True	True	True
stress-frontend-cpu-001	causality_only	cpu__checkoutservice	9	False	False	False
stress-frontend-cpu-001	kpiroot_combined	cpu__frontend	1	True	True	True
stress-paymentservice-cpu-001	similarity_only	cpu__paymentservice	1	True	True	True
stress-paymentservice-cpu-001	causality_only	memory__shippingservice	9	False	False	False
stress-paymentservice-cpu-001	kpiroot_combined	cpu__paymentservice	1	True	True	True

消融实验显示，在本项目较短的课程实验时序数据上，单独使用 Granger 风格因果得分并不稳定；仅使用 SAX-Jaccard 相似度和使用 KPIRoot 综合得分均能稳定命中真实根因。该现象与原论文中“相似度和因果分析均有贡献”的结论并不矛盾，因为原论文使用的是更长、更复杂的工业数据，而本项目故障窗口较短、故障类型也更直接。

从消融结果看，similarity_only 与 kpiroot_combined 在三组数据中都能命中真实服务，而 causality_only 的 Top-1 均偏离真实根因。这主要来自课程实验数据的两个特点：第一，故障窗口较短，PAA 后可用于 Granger 回归的点数有限；第二，Prometheus 的采样与 Kubernetes 恢复过程会使部分非根因指标在短窗口内出现统计上的先后关系。综合得分中相似度权重较高，因此能够抑制这类短序列因果噪声。

7 差异、局限与优化方向

- 原论文使用 Cloud H 工业环境的大规模主机集群/VM KPI，本项目使用 Online-Boutique 微服务级 KPI，因此数据规模和实体层级不同。
- 原论文的 alarm KPI 来自真实云系统告警，本项目使用聚合 CPU、聚合内存和前端探针指标模拟系统级告警。
- 原论文自动检测异常段，本项目优先使用 ChaosMesh 故障注入时间窗口，自动趋势检测作为备用逻辑。
- Pod Kill 的 Pod 身份变化在服务级宽表中被部分合并，后续可以保留 Pod 级 KPI 或引入事件日志，以提升该类故障的可解释性。
- 当前复现主要验证服务级根因定位，尚未引入调用链、日志或事件数据。若后续扩展，可将指标数据与 Kubernetes 事件、服务调用关系结合，提高对 Pod Kill 和网络延迟类故障的解释能力。

8 参与者与贡献

参与者	学号	本报告对应工作
王新杰	2311901	参与 ISSRE24-KPIRoot 阅读、环境部署、监控采集、故障数据整理、算法复现与报告整理。
王子祺	2312385	参与 ISSRE24-KPIRoot 阅读、复现方案讨论、结果校核与报告整理。

9 总结

本报告完成了 Online-Boutique 微服务系统部署、Prometheus/Grafana 监控、ChaosMesh 故障注入、Selenium/JMeter 测试以及 ISSRE24-KPIRoot 根因定位算法复现。实验结果表明，经过阶段二采集和处理的 KPI 数据能够支撑 KPIRoot 的核心流程，复现实现可以在三组故障场景中将真实根因服务排在第一位。与原论文相比，本项目的数据规模较小、场景经过课程实验简化，但已经覆盖了 PAA、SAX、相似度分析、因果得分、综合排序和消融实验等关键环节，并展示了该类监控指标根因定位方法在微服务实验环境中的可迁移性与局限性。

参考资料

- Wenwei Gu et al. KPIRoot: Efficient Monitoring Metric-based Root Cause Localization in Large-scale Cloud Systems. ISSRE 2024.
- JoinFyc/Online-Boutique: <https://github.com/JoinFyc/Online-Boutique>
- 项目仓库: <https://github.com/Sinclair987/software-testing-final-project>
- 软件测试与维护（2026年春）大作业要求。